PRACTICAL WEEK-1

Task1: Develop a C program that demonstrates how a Linux operating system executes a command entered by a user by accepting a Linux command as input, creating a child process using the fork() system call, executing the command in the child process using an appropriate exec() system call, allowing the parent process to wait for the child using wait(), and displaying the Process ID (PID) of both the parent and child processes.

Source code:

```
  GNU nano 7.2
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    char command[100];
    char *args[20];
    char *token;
    int i = 0;

    printf("Enter a Linux command: ");
    fgets(command, sizeof(command), stdin);

    // Remove newline
    command[strcspn(command, "\n")] = '\0';

    // Split command into arguments
    token = strtok(command, " ");

    while (token != NULL && i < 19) {
        args[i++] = token;
        token = strtok(NULL, " ");
    }

    args[i] = NULL;

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("\nChild Process\n");
        printf("Child PID  : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());
        printf("Executing command...\n");

        execvp(args[0], args);

        execvp(args[0], args);

        perror("exec failed");
        exit(1);
    }
    else {
        printf("\nParent Process\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID  : %d\n", pid);
        printf("Parent waiting for child...\n");

        wait(NULL);

        printf("Child process completed.\n");
        printf("Parent process completed.\n");
    }

    return 0;
}
```

Output:

```
avulasanjana@SanjanaReddy:~$ cd ~
nano week2pt1.c
avulasanjana@SanjanaReddy:~$ gcc week2pt1.c -o week2pt1
avulasanjana@SanjanaReddy:~$ ./week2pt1
Enter a Linux command: ls

Parent Process
Parent PID: 550
Child PID : 553
Parent waiting for child...


Child Process
Child PID : 553
Parent PID: 550
Executing command: ls

File1.txt            parser_double_quotes    session5_task2
File2.txt            parser_double_quotes.c  session5_task2.c
LICENSE              parser_quotes           session6_task1
Makefile             parser_quotes.c         session6_task1.c
OSSP                 process                 session6_task2
README.md            process.c               session6_task2.c
assets               process.c.save          session7_task1
bin                  scripts                 session7_task1.c
command_executor     session2_task1          session7_task2
command_executor.c   session2_task1.c        session7_task2.c
docs                 session2_task2          session8_task1
file_copy            session2_task2.c        session8_task1.c
file_copy.c          session3_task1          session8_task2
fork_exec            session3_task1.c        session8_task2.c
fork_exec.c          session3_task2          skill_week1_task2
include              session3_task2.c        skill_week1_task2.c
keyboard_input       session4_task1          src
keyboard_input.c     session4_task1.c        tests
main_loop            session4_task2          week2pt1
main_loop.c          session4_task2.c        week2pt1.c
obj                  session5_task1
ossp_2520090159      session5_task1.c
Child process completed.
Parent process completed.
avulasanjana@SanjanaReddy:~$
```

Observation: I learned how the Linux operating system creates a child process using fork() and executes a Linux command using the execvp() system call. I also understood how the parent process waits for the child process using wait() and how PID and PPID help identify the relationship between parent and child processes.

Task2: Using Linux terminal commands such as uname, lscpu, lsblk, ps, and top, investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the operating system abstracts and manages CPU, memory, storage, and I/O devices.

Output:

Using command uname -a:

```
avulasanjana@SanjanaReddy:~$ uname -a
Linux SanjanaReddy 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC
Thu Jun 18 21:54:43 UTC 2026 x86_64 GNU/Linux
```

Using command lscpu:

```
avulasanjana@SanjanaReddy:~$ lscpu
Architecture:                x86_64
  CPU op-mode(s):            32-bit, 64-bit
  Address sizes:             39 bits physical, 48 bits virtual
  Byte Order:                Little Endian
CPU(s):                      16
  On-line CPU(s) list:       0-15
Vendor ID:                   GenuineIntel
  Model name:                13th Gen Intel(R) Core(TM) i7-13620H
    CPU family:              6
    Model:                   186
    Thread(s) per core:      2
    Core(s) per socket:      8
    Socket(s):               1
    Stepping:                2
    BogoMIPS:                5836.80
    Flags:                   fpu vme de pse tsc msr pae mce cx8 apic sep mtr
                             r pge mca cmov pat pse36 clflush mmx fxsr sse s
                             se2 ss ht syscall nx pdpe1gb rdtscp lm constant
                             _tsc rep_good nopl xtopology tsc_reliable nonst
                             op_tsc cpuid tsc_known_freq pni pclmulqdq vmx s
                             sse3 fma cx16 pcid sse4_1 sse4_2 x2apic movbe p
                             opcnt tsc_deadline_timer aes xsave avx f16c rdr
                             and hypervisor lahf_lm abm 3dnowprefetch ssbd i
                             brs ibpb stibp ibrs_enhanced tpr_shadow ept vpi
                             d ept_ad fsgsbase tsc_adjust bmi1 avx2 smep bmi
                             2 erms invpcid rdseed adx smap clflushopt clwb
                             sha_ni xsaveopt xsavec xgetbv1 xsaves avx_vnni
                             vnmi umip waitpkg gfni vaes vpclmulqdq rdpid mo
                             vdiri movdir64b fsrm md_clear serialize ibt flu
                             sh_l1d arch_capabilities
Virtualization features:
  Virtualization:            VT-x
  Hypervisor vendor:         Microsoft
  Virtualization type:       full
Caches (sum of all):
  L1d:                       384 KiB (8 instances)
  L1i:                       256 KiB (8 instances)
  L2:                        10 MiB (8 instances)
  L3:                        24 MiB (1 instance)
```

```
NUMA:
  NUMA node(s):              1
  NUMA node0 CPU(s):         0-15
Vulnerabilities:
  Gather data sampling:      Not affected
  Ghostwrite:                Not affected
  Indirect target selection: Not affected
  Itlb multihit:             Not affected
  L1tf:                      Not affected
  Mds:                       Not affected
  Meltdown:                  Not affected
  Mmio stale data:           Not affected
  Old microcode:             Not affected
  Reg file data sampling:    Mitigation; Clear Register File
  Retbleed:                  Mitigation; Enhanced IBRS
  Spec rstack overflow:      Not affected
  Spec store bypass:         Mitigation; Speculative Store Bypass disabled v
                             ia prctl
  Spectre v1:                Mitigation; usercopy/swapgs barriers and __user
                              pointer sanitization
  Spectre v2:                Mitigation; Enhanced / Automatic IBRS; IBPB con
                             ditional; PBRSB-eIBRS SW sequence; BHI BHI_DIS_
                             S
  Srbds:                     Not affected
  Tsa:                       Not affected
  Tsx async abort:           Not affected
  Vmscape:                   Not affected
```

Using command lsblk:

```
avulasanjana@SanjanaReddy:~$ lsblk
NAME MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda    8:0    0 356.9M  1 disk
sdb    8:16   0 159.4M  1 disk
sdc    8:32   0     2G  0 disk [SWAP]
sdd    8:48   0     1T  0 disk /mnt/wslg/distro
                                /
```

Using command ps:

```
avulasanjana@SanjanaReddy:~$ ps
    PID TTY          TIME CMD
    636 pts/2    00:00:00 bash
    702 pts/2    00:00:00 ps
```

Use command top:

```
avulasanjana@SanjanaReddy:~$ top
top - 15:12:20 up 39 min,  1 user,  load average: 0.07, 0.04, 0.06
Tasks:  32 total,   1 running,  31 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.
MiB Mem :   7771.7 total,   7077.0 free,    605.5 used,    257.7 buff/cache
MiB Swap:   2048.0 total,   2048.0 free,      0.0 used.   7166.1 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+
   1905 root      20   0   33916   9500   7720 S   0.3   0.1   0:00.66
      1 root      20   0   24608  14356  10124 S   0.0   0.2   0:02.93
      2 root      20   0    3180   2208   2072 S   0.0   0.0   0:00.01
      6 root      20   0    3180   2048   1940 S   0.0   0.0   0:00.00
    156 root      20   0    2888   1944   1820 S   0.0   0.0   0:00.04
    157 root      20   0    4472   3156   2908 S   0.0   0.0   0:00.01
    158 message+  20   0    9100   5792   4696 S   0.0   0.1   0:00.52
    171 root      20   0   44092  29672  14532 S   0.0   0.4   0:00.17
    185 root      20   0   18648   9476   8144 S   0.0   0.1   0:00.19
    235 _chrony   20   0   24472  11888   7084 S   0.0   0.1   0:00.13
    251 _chrony   20   0   12092   2464   1816 S   0.0   0.0   0:00.01
    283 root      20   0  123456  32632  18020 S   0.0   0.4   0:00.14
    286 root      20   0    5492   2760   2572 S   0.0   0.0   0:00.00
    338 root      20   0    3184   1112    980 S   0.0   0.0   0:00.00
    339 root      20   0    3200   1248   1108 S   0.0   0.0   0:00.27
    340 avulasa+  20   0    6268   5528   3860 S   0.0   0.1   0:00.05
    341 root      20   0    8644   5524   4684 S   0.0   0.1   0:00.01
    384 avulasa+  20   0   22480   9600   7160 S   0.0   0.1   0:00.26
    386 avulasa+  20   0   22908   4088   2088 S   0.0   0.1   0:00.00
    416 avulasa+  20   0    6136   5440   3864 S   0.0   0.1   0:00.03
    634 root      20   0    3184   1120    980 S   0.0   0.0   0:00.00
    635 root      20   0    3200   1256   1108 S   0.0   0.0   0:00.04
    636 avulasa+  20   0    6268   5536   3864 S   0.0   0.1   0:00.04
    714 avulasa+  20   0    8296   6096   3900 R   0.0   0.1   0:00.95
   1031 polkitd   20   0  380176   8640   7704 S   0.0   0.1   0:00.13
   1842 root      19  -1   50512  14632  13408 S   0.0   0.2   0:00.10
   1969 systemd+  20   0   22544  11356   8736 S   0.0   0.1   0:00.05
   2433 avulasa+  20   0    8544   5004   4584 S   0.0   0.1   0:00.00
  10430 syslog    20   0  220548   5172   4348 S   0.0   0.1   0:00.03
  12358 root      20   0 1792300  14720  11904 S   0.0   0.2   0:00.08
  12452 root      20   0   33920   5160   3344 S   0.0   0.1   0:00.00
  12453 root      20   0   33920   5160   3344 S   0.0   0.1   0:00.00
```

Observation: I learned how to use Linux commands to examine system hardware and running processes. I understood how the operating system manages CPU, storage, memory, and processes and provides these resources to applications through system services